Data Visualization MDS/MEI

Second practical work

Introduction

The goal of this project is to create a visual analytics tool that lets you explore the words that have been issued by the main characters of the Simpsons throughout the episodes.

You can use the same dataset as proposed for the first practical work, and take the scripts file.

With the provided data, you need to create an application that lets the user answer the following questions using Python and Vega-Altair that answer to the following questions:

- What are the characters that issue more words, and how are the word numbers distributed?
- How have the word numbers among the characters evolved throughout the available seasons?
- I would like to compare the word distribution for a pair of concrete characters, for a selected season, and, in addition, for a selected episode.
- I would like the same comparison as before, but over a concrete episode.
- I would also like to answer the first question, but regarding sentences, not words.

Most of these questions require interactions, otherwise the charts will not be easy to interpret. The final visualization must include multiple views. Cross interactions are a must, that is, when selecting a character, any of the views that are showing the character must react to the selection. How, it is up to you.

Like in the previous case, the final vis must also be implemented using Streamlit.

You may add extra questions.

Think carefully the design of all the charts, including the final visualization. Test and redesign.

Data

Dataset: https://www.kaggle.com/datasets/prashant111/the-simpsons-dataset/data. This Kaggle website contains a set of files. For the moment, we are interested in the chapters' ratings, that are stored in the CSV file called *simpsons_episodes.csv*. Check the files' descriptions to understand what they contain.

You need to clean it and keep relevant information if needed.

Data processing

The data cleaning can be done offline. You do not need to explain it in this project.

Design and implementation

For each question, an interactive chart (or more than one) needs to be designed and implemented. You may tackle the problem in different ways: there is not a single solution. Ensure that the answers are evident (maybe after interaction). You also need to provide a text of a maximum of 200 words per question (and for the final visualization too) where you explain the design decisions and how these help the users to answer the questions properly. This should include aspects such as: what type of chart did you select and why, what were the different steps of design process you followed, what changes you applied to improve legibility, reduce clutter,

distinguish elements, how does the interaction allow us to answer the questions, etc. You need to discuss other alternatives you considered (or implemented) that did not work. Be concise. You do not need to erase the previous steps in design you followed; they are actually encouraged. But only a final description per task is necessary.

Remember, a final visualization is also required that includes all questions. In this case, you also have to explain, in a maximum of 200 words, what changes were made to make charts consistent, aesthetically pleasant, etc. The final visualization must be then incorporated in a Streamlit application (https://streamlit.io/).

Delivery instructions

The work can be implemented individually or in pairs.

The delivery must consist on a single ZIP file with a name that includes the authors, that contains the datasets (raw and clean), the Colab file(s) (*ipnyb*), the Python Streamlit application, and optional extra documents if required. The ZIP, Streamlit, and Colab files must be named after the names of the authors. Treat the Colab document as a report, include titles, boldfaces, etc., to make it easier to read.

Upon delivery, an interview with the teacher of your lab will be held to help us understand your development. Such an interview is compulsory; it is part of the delivery. During the interview, the teacher will ask you questions about the project, such as design decisions, or implementation details. How you answer those questions will determine your individual grade. These interviews will happen during a regular lab session, or in the teacher's office. They will likely be individual, independently of whether the project was done in pairs or not.

The deadline for the delivery of this lab project is the 31st of May. Delivery through Racó (raco.fib.upc.edu).

Important remarks

The final grade will take into account the number of cross interactions included in the visualizations (that are used to effectively solve tasks). Additionally, we will value the number of non-trivial tasks (adequately described in the documentation) that can be properly solved with your visualization tool. In this sense, adding other data sources if suitable will be appreciated.

Don't leave the project for the last day or do the minimum amount of work. In case of doubt, ask whether the current work is enough or needs more effort.

This checklist serves as guidance for your delivery

Global checklist:

- Name the file after the name(s) of the author(s).
- Include the name(s) of the authors also as the first line in your notebook.
- Include the clean data.
- Compress all files in a single zip (do not use RAR or other formats) file.
- Also include the names of the authors in the notebook (e.g., a text cell showing who authored the document) and in the Streamlit app (e.g., in an "About" option).
- Ensure the names of the data files inside the notebook correspond to the ones you deliver.
- Make a single delivery per group.
- Ensure you properly cleaned the data.
- Ensure the code executes without errors: last-minute changes may lead to typos.

- Ensure you include a step-by-step design process (does not need to include all minimal steps, if you prefer, but do not forget to include the discussion on why do you change something).
- Do not mix charts with other technologies, everything must be created using altair.
- Ensure they do not include non-properly cleaned data (e.g., N/A or undefined fields).

Google Colab document. For every chart, you must consider:

- Color blindness (e.g., coding anything just with a red-green palette may be problematic).
- Check the consistency of colors across the whole visualization (same color, same meaning in different charts).
- Do not forget to add meaningful titles, labels, messages if necessary...
- When using colors with opacity different from zero, check the interactions with the other elements (are they visible?).
- Think whether you need to normalize values.

For the Streamlit vis:

- All questions should be solved in a single page (with not much scrolling, e.g., maximum double the available screen on a 15" laptop screen).
- The application should run with a simple "streamlit run <application_name>" in any computer, everything should be available in the same folder or addressed properly.
- Do not forget to include a final visualization that answers all the questions.
- Align things, be consistent. You can make use of both vertical and horizontal alignments to facilitate comparisons.
- Extra questions you answer must also go into the final vis.
- Ensure charts can be visually compared properly (consider changing scales, palettes...)
- Use space cleverly (put related things together and unrelated things far away).
- The higher the number of variables (and questions answered) included, the better.
- If you do not use all your data in your vis, ensure you have filtered it previously, don't force Streamlit to execute with data it is not used.
- **Ensure the default values make sense (e.g., if including selections, check that the initial configuration of the chart has a default state that is meaningful).**
- **Ensure that interactions do not render charts useless, e.g., leaving charts blank, or with NaNs.**
- **Minimize the number of interactions required to solve the problems.**